Retrospective: Coroutine Executors and P2464R0

Document Number:	P4062R0
Date:	2026-03-14
Audience:	LEWG, SG1

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure
   - P2464R0
   - P2300R10
2. History
   - 2.1 The Evolution of Scope
   - 2.2 The Evolution of Terminology
   - 2.3 The Two Framings
3. The Three Criteria
4. The Error Channel
   - 4.1 The Lost Framing
   - 4.2 The Coroutine Model
   - 4.3 The Downstream Consequence
   - 4.4 The Causal Chain
   - 4.5 The Steel Man
5. Lifecycle
6. Composition
   - 6.1 What the Quote Means
   - 6.2 The Two Axes
   - 6.3 Complexity and Evidence

6.4 The Two Framings in Code
6.5 The Original Diagnosis
7. Structured Concurrency
8. The Executor Is Application Policy
Acknowledgments
References

Abstract

P2464R0[1], "Ruminations on networking and executors," identified three properties of P0443R14[3], "A Unified Executors Proposal for C++," executors: no error channel, no lifecycle for submitted work, and no generic composition. This paper checks whether those properties hold for the coroutine-native executor described in P4003R0[2], "Coroutines for I/O." They do not. It also re-examines whether the original diagnosis of P0443R14[3] was correct. The diagnosis was a downstream consequence of the scope expansion and terminology shift documented in Section 2. The coroutine executor concept did not exist in its current form until P4003R0[2] was published in 2026; this analysis became possible only then. Section 2 traces the scope expansion and terminology shift that created P0443R14[3] from three independent executor models and erased the continuation framing from the API surface. Sections 3-6 evaluate the coroutine executor against P2464R0[1]'s criteria. Sections 7-8 document structured concurrency and the executor concept.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

1. Disclosure

The authors developed and maintain Corosio[5] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The authors provide information, ask nothing, and serve at the pleasure of the chair.

The authors are revisiting the historical record systematically. This paper is one of several. The goal is to understand - precisely and on the record - every decision that kept networking out of the C++ standard. That effort requires re-examining consequential papers, including papers written by people the authors respect. Some of the conclusions are uncomfortable. But the committee has endured far greater discomfort from the consequences of decisions made with good intentions and incomplete evidence. If this work helps inform people in a way they find useful, the discomfort is worth bearing.

P2464R0

P2464R0[1] was consequential. It provided the analytical framework that led the committee to set aside the Networking TS and focus on the sender/receiver model. Decisions of that magnitude deserve periodic review. This paper provides one. Where the analysis finds that a claim does not hold - whether applied to the coroutine executor or to the original P0443R14[3] model - the authors say so. The intent is to ensure the committee's record is accurate, not to assign blame.

P2300R10

P2464R0[1] critiqued P0443R14[3]. P2300R10[6], "std::execution," addressed those deficiencies with the sender/receiver model. P4003R0[2] provides a second async model. The committee now has two implementations to evaluate against P2464R0[1]'s criteria. This paper evaluates P4003R0[2] and uses P2300R10[6] as a reference. Structural comparisons are observations, not arguments that `std::execution` should be modified or removed.

---

2. History

The executor that P2464R0[1] critiqued was not the executor that Kohlhoff built. Two independent causal chains - a scope expansion and a terminology shift - converged to erase the original design intent from the API surface. By the time P2464R0[1] was written, the only framing visible was what this paper calls the work framing. P2464R0[1]'s conclusions are internally consistent under that framing. They do not hold under the original framing - what this paper calls the continuation framing. The following subsections document both chains.

2.1 The Evolution of Scope

Three independent executor models existed by 2014, each deployed in its domain. The committee directed the authors to unify them into a single abstraction. The published record contains a preference and a direction. It does not contain evidence that unification would work.

Step	Year	What happened	Evidence
Three models	2014	Kohlhoff (networking), Hoberock/Garland (GPU), Mysen (thread pools)	Each deployed, each working
SG1 Redmond	2014	"Start with Mysen's proposal" - directed to unify (N4199[21])	Straw poll, no prototype
P0443R0[20]	2016	"Unifies three separate executor design tracks"	No prototype, no experiment
P0761R2[22]	2018	Design document published	No analysis of domain loss
P0443R14[3]	2020	Final unified proposal	Never deployed as unified

What the published record does not contain:

- No analysis of whether networking, GPU dispatch, and thread pools ***can*** share a single abstraction without losing domain semantics.
- No prototype demonstrating that a unified concept works for all three domains.
- No experiment comparing unified and domain-specific approaches.
- No discussion of what each domain would lose in the unification.
- No evaluation of whether the `parallel_for` dispatch problem - an algorithm selecting a facility - is the same problem as a networking library scheduling continuations.

Kohlhoff described what happened in P1791R0[19]:

> *"We all saw our own requirements as essential and therefore universally applicable. As a result initial compromises were focused on gaining buy-in from other parties that those requirements were in fact universal and should be accepted as such. A significant part of the consensus building experience was to realise that while they may be essential in a particular domain, they were not universal."*

WG21 is where great ideas go to become good ideas (Falco). While Falco is known for sharp phrasing, this one is a reminder, not a criticism - consensus is how C++ ships; yet consensus without deployment evidence drifts toward the average, and the average is not where we want to be. Three working models were replaced by one that had never been deployed. The domain-specific requirements that made each model correct for its domain were stripped because they were not universal. The result served no domain as well as the originals served theirs.

2.2 The Evolution of Terminology

Independently of the scope expansion, the terminology shifted. What began as continuation-scheduling primitives was progressively renamed, demoted, and erased until the continuation framing was no longer visible on the API surface.

Paper	Year	Continuation status
P0113R0[10]	2015	First-class: `defer` = "continuation of the caller"
P0688R0[11]	2017	Demoted to `prefer(is_continuation)` hint
P0761R2[22]	2018	Optional property, framed as "may execute more efficiently"
P1525R0[13]	2019	Pure work language. One incidental mention of "continuation"
P1660R0[12]	2019	"Eagerly submits work." Executor = "factory for execution agents"
P0443R14[3]	2020	Section heading: "Executors Execute Work." Continuation property gone
P2464R0[1]	2021	"Work-submitter." Credits "work-executing model." No `async_result`, no N3747[14]

By 2021, the continuation framing had been erased from the API surface. The work framing was all that remained. Each step was locally reasonable. The cumulative effect was that a careful reviewer, analyzing the API as presented, could not see the continuation semantics that had been stripped. The terminology determined the analysis. The committee renamed continuation-scheduling to `execute`, lost the continuation semantics, and then misinterpreted the result for lacking those semantics.

Under the work framing, the executor is the composition mechanism - and it has no error channel, no lifecycle, no generic composition. Under the continuation framing, the executor is a scheduling policy and `async_result` is the composition mechanism. P2464R0[1] never mentions `async_result`, completion tokens, or N3747[14]. The work framing made them invisible.

2.3 The Two Framings

The continuation framing. `dispatch` / `post` / `defer` schedule a continuation on an execution context. The callable is a resumption handle. The operating system performs the work. The result is delivered to the continuation when it wakes up. The executor never touches the result. This is where we were.

The work framing. `execute(F&&)` submits work. The callable is a unit of work. The executor runs it. If dropped, the work and its result are lost. Error handling, lifecycle, and composition are the executor's responsibility. This is what it became.

Sections 4-6 evaluate P2464R0[1]'s claims under both framings.

---

3. The Three Criteria

P0443R14[3]:

```
void execute(F&& f);
```

Coroutine executor (P4003R0[2]):

```
std::coroutine_handle<> dispatch(
    std::coroutine_handle<> h) const;

void post(std::coroutine_handle<> h) const;
```

P2464R0[1] identified three deficiencies in P0443R14[3] that made it inadequate as a foundation for async programming:

1. No error channel.
2. No lifecycle for submitted work.
3. No generic composition.

P2300R10[6] addressed all three with the sender/receiver model. A coroutine executor is a type satisfying the `Executor` concept described in P4003R0[2]. This paper checks whether the three properties hold for `dispatch(coroutine_handle<>)` and `post(coroutine_handle<>)`, and re-examines whether they held for `execute(F&&)`.

---

4. The Error Channel

P2464R0[1] argued that P0443R14[3]'s `execute(F&&)` has no error channel at the call site, and that errors after the call are an "irrecoverable data loss." This section addresses both claims.

4.1 The Lost Framing

P2464R0[1]:

> *"The only thing that knows whether the work succeeded is the same facility that accepted the work submission."*

Three terms. All three are downstream consequences of the work framing that the rename created. None of them hold under the continuation framing.

1. "The work." Under the continuation framing, there is no work. The operating system performs the I/O. The callable is a continuation that resumes after the OS completes. It is not work. It is the opposite of work - it is the thing that was suspended while the work happened.
2. "Succeeded." Under the continuation framing, a continuation does not succeed or fail. It resumes. The I/O result - error code, byte count - is delivered to the continuation when it wakes up. The continuation did not produce the result. The OS did.
3. "Accepted the work submission." Under the continuation framing, no work was submitted. The caller yielded control. As documented in Section 2, Kohlhoff's original API named this `dispatch`, `post`, and `defer` - continuation-scheduling primitives. The committee renamed them to `execute`, reframing a yield as a submission.

P2464R0[1] analyzed the renamed API under the work framing. The use cases it describes - queue full, executor shutting down - presuppose a fire-and-forget model where a caller hands off work and moves on. The original API did not work that way. The rename obscured this.

The authors accepted the work framing too. It was not until `dispatch` and `post` reappeared in the coroutine executor - taking `coroutine_handle<>`, not `F&&` - that the distinction resurfaced.

4.2 The Coroutine Model

The coroutine model respects the original semantics because structurally it has no choice. `co_await` suspends the caller. It is not running. It does not need an error channel back because it is not alive to act on one. It will either be resumed with a result or destroyed, propagating cancellation. There is no state where the caller is

alive, running, and uninformed. `coroutine_handle<>` is manifestly a suspended continuation, not a unit of work. The fire-and-forget use cases do not arise. Coroutines are not fire-and-forget. Applying the fire-and-forget analysis to a suspension-based model is a category error.

4.3 The Downstream Consequence

P2464R0[1] said P0443R14[3] executors "toss error information into a lake."

Under the continuation framing, there is no error information. At the point of the `execute` call, the I/O has not completed. The operating system has not returned a result. There is no error code. There is no byte count. The only thing being passed to the executor is a resumption handle: this is how you resume me. That is all a callback is. That is all a future is. That is all a completion token is. That is all a `coroutine_handle<>` is.

If the executor drops the handle, the loss is not "error information." The loss is the resumption itself. The caller will never wake up. That is a real problem - but it is not the problem P2464R0[1] described. P2464R0[1] described information loss. The actual problem is continuation loss. These are different problems with different solutions.

The distinction between information loss and continuation loss is real. Under the work framing, the distinction is invisible - if `execute` submits work, then losing the work loses its result. Under the continuation framing, the distinction is obvious - a continuation carries no result because the result does not yet exist. The framing determined the diagnosis. The framing was a downstream consequence of the steps documented in Section 2.

4.4 The Causal Chain

Trace the path of an asynchronous read in the Networking TS:

1. The caller initiates `async_read(socket, buffer, handler)`. The implementation stores the handler - a completion token - and begins the I/O. The handler means: this is how you resume me.
2. The operating system performs the read. The caller is not running. The handler is not running. The OS is doing the work.
3. The OS completes. The implementation now holds the result: an error code and a byte count. It calls `executor.dispatch(bound_handler)`, where `bound_handler` carries the result and the original handler together. In the committee's renamed API, this became `executor.execute(bound_handler)`.
4. The executor resumes the handler on the correct execution context. The handler receives the error code and the byte count.

At no point in this chain is error information "tossed into a lake." The result does not exist at step 1. The result is created by the OS at step 3. The result is bound into the handler before the executor sees it. The executor never touches the result. It never inspects it. It never drops it. The executor's only job is to resume the handler on the right context. The result travels inside the handler, not alongside it.

Under the work framing, this chain looks like information loss - the executor receives work and its result, and if the executor drops it, the result is lost. Under the continuation framing, the chain looks different: the executor receives a resumption handle, the result does not yet exist, and there is nothing to lose. Both readings are internally consistent. They differ because the framing differs. The rename in Section 2.2 determined which framing was visible.

4.5 The Steel Man

The strongest form of P2464R0[1]'s claim: the OS completes the I/O, the result exists, and the executor drops the continuation. What happens in each model?

Networking TS:

```
async_read(socket, buffer,
    [](error_code ec, size_t n) {
        process(ec, n);
    });
// caller returns here - gone
```

The caller returned at the call site. The caller is gone - back to the event loop or up the call stack. If the executor drops the bound handler, a future action does not happen. Nobody is waiting. Nobody is hung. Nobody lost information they were holding. The result was created by the OS, bound into the handler, and destroyed with the handler. It was never delivered to anyone. This is a dropped packet, not a lost letter.

Coroutine model:

```
auto [ec, n] =
    co_await socket.read_some(buffer);
// coroutine is suspended
```

The coroutine is suspended. The ownership contract requires the executor to resume or destroy. If it destroys the handle, the coroutine frame is destroyed, the parent is notified via destruction propagation. Nobody is hung. The result was never delivered but the coroutine was asleep - you cannot lose information you never received.

`sync_wait` is a testing and bridging utility, not a production async pattern. This comparison is limited to models where the initiator is asynchronous.

Model	Initiator state	Result	Program state
Networking TS	Gone (returned)	Destroyed with handler	Not hung
Coroutine model	Suspended	Destroyed with operation state	Not hung (ownership contract)

P2464R0[1] used "information loss" to reject the Networking TS. The Networking TS cannot lose information because the initiator returned - there is nobody to lose it to. The coroutine model cannot hang because the ownership contract forces cleanup. Neither model leaves a thread alive, waiting, and permanently uninformed.

---

5. Lifecycle

P2464R0[1]:

> *"A P0443 executor is not an executor. It's a work-submitter."*

The work framing is inconsistent with the continuation framing. Under the continuation framing, the callback is a continuation - it is how the initiator resumes after the OS completes. The executor takes it and resumes it on a context. That is all it does.

> *"Trying to entertain the fantasy that the work submission and result observation are separable concepts is just **wrong**."*

Under the continuation framing, work submission and result observation are not separable because they are not separate things. The executor takes a continuation and resumes it on a context. The work is done by the operating system. The result is delivered ***to*** the continuation when it resumes - result observation is not separable from the continuation because the result arrives inside it. P2464R0[1] diagnosed a lifecycle problem under the work framing. Under the continuation framing, the problem does not arise.

The coroutine model makes the lifecycle explicit.

`post(coroutine_handle<>)` has two call-site dispositions:

- **Normal return.** Ownership transferred. The executor holds the handle and is responsible for it.
- **Exception.** Scheduling rejected. The caller still owns the handle.

After ownership is transferred, the execution context has two dispositions:

- **Resume.** The continuation runs.
- **Destroy without resuming.** The awaiting coroutine is also destroyed, propagating cancellation upward through the coroutine tree. This is analogous to `set_stopped` in P2300R10[6].

The executor holds a typed resource with a defined lifecycle. It is not an opaque callable. A `coroutine_handle<>` has exactly two operations: `resume()` and `destroy()`. The proposed wording specifies the ownership contract:

```
std::coroutine_handle<> dispatch(
    std::coroutine_handle<> h) const;

void post(std::coroutine_handle<> h) const;
```

Preconditions: `h` is a valid, suspended coroutine handle.

Effects: Arranges for `h` to be resumed on the associated execution context. For `dispatch`, the implementation may instead return `h`. For `post`, the coroutine is not resumed on the calling thread before `post` returns.

Postconditions: For `post`, ownership of `h` is transferred to the implementation. For `dispatch`, if `noop_coroutine()` is returned, ownership of `h` is transferred to the implementation; if `h` is returned, ownership of `h` is transferred to the caller via the return value and the implementation has no further obligation regarding `h`. When ownership is transferred to the implementation, the implementation shall either resume `h` or destroy `h`. No other disposition is permitted.

Returns (*`dispatch`* ***only):*** `h` or `noop_coroutine()`.

Throws: `std::system_error` if scheduling fails. If an exception is thrown, ownership of `h` is not transferred; the caller retains ownership.

[Note: The returned handle enables symmetric transfer. The caller's `await_suspend` may return it directly. ***- end note]***

When the executor destroys a queued handle, the awaiting coroutine's `await_resume` is never called. The executor provides the ownership contract: resume or destroy, no other disposition. Structured concurrency combinators provide the propagation guarantee: if the parent coroutine is itself suspended in `when_all` or a similar combinator, the combinator detects the child's destruction and resumes the parent with an error. If no combinator is involved, the parent is destroyed in turn. Both guarantees are needed; neither alone is sufficient. Together, cancellation propagates upward through the coroutine tree. No handle is leaked. No information is lost. The propagation guarantee depends on the promise type contract; `task<>` enforces it. Raw `coroutine_handle<>` users are in the same position as raw pointer users - the abstraction provides safety, the escape hatch does not.

---

6. Composition

P2464R0[1]:

> *"How many different implementations of this function do I need to support 200 different execution strategies and 10,500 different continuation-decorations? With Senders and Receivers, the answer is 'one, you just wrote it.'"*

6.1 What the Quote Means

"Execution strategies" means different executors - thread pools, event loops, strands, GPU contexts. The axis of ***where*** the continuation resumes.

"Continuation-decorations" means different completion models - callbacks, futures, coroutines, `use_awaitable`. The axis of ***how*** the initiator receives the result.

The claim: if you have M execution strategies and N completion models, you need M x N implementations of each I/O operation unless you have generic composition. With senders and receivers, you write it once.

6.2 The Two Axes

P2464R0[1]'s composition claim conflates two independent axes.

The completion-model axis is the axis of ***how*** the initiator receives the result - callback, future, coroutine, or fiber. N3747[14], "A Universal Model for Asynchronous Operations" (Kohlhoff, 2013), solved this axis eight years before P2464R0[1] was written. The mechanism is `async_result`: each completion token specializes

`async_result` once, and every async operation works with it. One function signature serves every completion model:

```
template<class ReadHandler>
DEDUCED async_read(
    AsyncReadStream& s,
    DynamicBuffer& b,
    ReadHandler&& handler);
```

The same `async_read` works with a callback, a future, or a coroutine. M executors and N completion models require one implementation of `async_read` plus N specializations of `async_result`. Not M x N. P2464R0[1] did not address `async_result` or N3747[14].

The operation-composition axis is the axis of chaining operations into higher-level protocols - composing `async_read_some` into `async_read`, composing socket reads into HTTP message parsing into WebSocket frames. On this axis, P2464R0[1] had a legitimate point. Each composed operation in the Networking TS required a hand-written state machine. Boost.Beast (Boost 1.66, 2017) deployed three layers of composed asynchronous operations - socket reads into HTTP parsing into WebSocket framing - and every layer required its own state machine, its own intermediate completion handler, and its own lifetime management. The composition worked. It was deployed. It scaled from low-level to high-level. But it was genuinely difficult to write, difficult to review, and difficult to get right. The authors wrote those state machines and can attest to the cost.

Coroutines solve the operation-composition axis. `co_await` replaces the state machine. A composed read that required a hundred-line state machine in the callback model is a ten-line coroutine body. The coroutine frame holds the state. The compiler manages the suspension points. The composition that was manual and error-prone becomes sequential and obvious. This is the forward-looking answer to P2464R0[1]'s composition concern - not that the concern was wrong, but that C++20 resolved it.

6.3 Complexity and Evidence

P2464R0[1]:

> *"Composing Networking TS completion handlers or asynchronous operations (which aren't even a thing in the NetTS APIs, so how would I be able to compose those other than in an ad-hoc manner?) seems like an ad-hoc and non-generic exercise. Whether they were designed for or ever deployed in larger-scale compositions that scale from low-level to high-level with any sort of uniformity, I don't know."*

The admission is honest, and honesty is admirable. The compositions existed. Asio's own `async_read` composes `async_read_some`. Boost.Beast (Boost 1.66, 2017) composes socket-level reads into HTTP message parsing into WebSocket frames - three layers of composed asynchronous operations, all using `async_result`, all scaling from low-level to high-level. Both were deployed before P2464R0[1] was written. "Which aren't even a thing in the NetTS APIs" - asynchronous operations are the Networking TS API. `async_read`, `async_write`, `async_connect`, `async_accept` - every one is a composed asynchronous operation built from lower-level operations using the universal model documented in N3747[14].

It should not surprise anyone that a reviewer found this mechanism opaque. Kohlhoff's universal async model - `async_result`, completion tokens, default completion token template parameters, the initiating function ceremony that prepares the result inside the operation body - is the most complex customization point the authors have encountered. The complexity that made it hard to learn was the same complexity that made it important to learn: it was the mechanism that answered P2464R0[1]'s composition question on the completion-model axis.

The gap was institutional. The universal async model was the most complex mechanism in the Networking TS, and its complexity made it opaque to reviewers who had not built on it.

The committee removed a Technical Specification from its work program. The consequences of that decision persist today. The analysis that informed it said, in its own words, "I don't know" whether the TS's asynchronous operations could compose. The compositions existed. They were deployed. They were documented in N3747[14]. A decision of that magnitude, informed by acknowledged uncertainty about a mechanism that was already shipping, could have prompted an investigation. It prompted a conclusion.

Under the work framing, there is no reason to look for `async_result` because the executor is the composition mechanism. The universal async model is part of the continuation framing. When the continuation framing was lost, `async_result` became invisible.

6.4 The Two Framings in Code

P2464R0[1]'s composition argument rests on pseudo-code. Compare it with the code it was analyzing:

The continuation framing (P0113R0, 2015)	The work framing (P2464R0, 2021)

```
// Pass the result to the handler,
// via the associated executor.
dispatch(work.get_executor(),
    [line=std::move(line),
     handler=std::move(handler)]
        () mutable
    {
        handler(std::move(line));
    });
```

```
void run_that_io_operation(
    scheduler auto sched,
    sender_of<network_buffer> auto
        wrapping_continuation)
{
    launch_our_io();
    auto [buffer, status] =
        fetch_io_results_somehow(
            maybe_asynchronously);
    run_on(sched,
        queue_onto(wrapping_continuation,
            make_it_queueable(
                buffer, status)));
}
```

On the left, the executor resumes the continuation with the result. The result travels inside the handler. The executor never touches it. On the right, the executor launches work, fetches results, and queues them. Every verb assumes the executor is doing the work.

Same problem. Two framings. Two completely different code shapes. The left is from the era when the continuation framing was visible. The right is from after the rename erased it. The pseudo-code is not wrong - it is a faithful expression of the work framing. It is evidence that the work framing had fully replaced the continuation framing by 2021.

6.5 The Original Diagnosis

P2464R0[1]:

> *"Composing Networking TS completion handlers or asynchronous operations ... seems like an ad-hoc and non-generic exercise."*

On the completion-model axis, the characterization did not account for `async_result`. `async_result` adapts any completion token to any async operation. The universal model was documented in N3747[14]. It was designed, documented, and deployed. P2464R0[1] did not address it.

On the operation-composition axis, the characterization had merit. Each composed operation required a hand-written state machine. The composition was deployed and it worked, but the difficulty was real. Coroutines resolve it.

The preceding sections address P2464R0[1]'s criteria. The following sections document additional properties of the coroutine constraint.

---

7. Structured Concurrency

```
capy::task<dashboard> load_dashboard(
    std::uint64_t user_id)
{
    auto [name, orders, balance] =
        co_await capy::when_all(
            fetch_user_name(user_id),
            fetch_order_count(user_id),
            fetch_account_balance(user_id));
    co_return dashboard{name, orders, balance};
}
```

```
capy::task<> timeout_a_worker()
{
    auto result = co_await capy::when_any(
        background_worker("worker"),
        capy::delay(100ms));

    if (result.index() == 1)
        std::cout << "Timeout fired\n";
}
```

---

8. The Executor Is Application Policy

P2464R0[1] treated thousands of executor implementations as a problem:

> *"C++ programmers will write **thousands** of these executors."*

It was the design intent. P0113R0[10] (Kohlhoff, 2015):

> *"Like allocators, library users can develop custom executor types to implement their own rules."*

Thousands of implementations is what you get when an abstraction works. P0285R0[15] (Kohlhoff, 2016):

> *"The central concept of this library is the executor as a policy."*

The coroutine executor concept (Capy[4]) makes this practical:

```
template<class E>
concept Executor =
    std::is_nothrow_copy_constructible_v<E> &&
    std::is_nothrow_move_constructible_v<E> &&
    requires(E const& ce, std::coroutine_handle<> h)
{
    { ce == ce } noexcept -> std::convertible_to<bool>;
    { ce.context() } noexcept;
    { ce.dispatch(h) } -> std::same_as<
        std::coroutine_handle<>>;
    { ce.post(h) };
};
```

Two operations. Both take a `coroutine_handle<>`. Both resume it on a context. Every executor speaks the same protocol. Most users never see it - they write `task<>` coroutines and pick an executor at the launch site. The concept exists for the people who build execution contexts, not the people who use them.

Acknowledgments

The authors thank Peter Dimov for identifying that P0443R14's callable destruction is detectable, correcting an earlier version of Section 4; Dietmar Kühl for `beman::execution` and ongoing work on P3552R3; Ville Voutilainen for P2464R0, which provided the evaluation framework for this paper; Steve Gerbino and Mungo Gill for Capy and Corosio implementation work; and Klemens Morgenstern for Boost.Cobalt and the cross-library bridge examples.

References

1. P2464R0 - ''Ruminations on networking and executors'' (Ville Voutilainen, 2021). https://wg21.link/p2464r0
2. P4003R0 - ''Coroutines for I/O'' (Vinnie Falco, Steve Gerbino, Mungo Gill, 2026). https://wg21.link/p4003r0
3. P0443R14 - ''A Unified Executors Proposal for C++'' (Jared Hoberock, et al., 2020). https://wg21.link/p0443r14
4. Capy - Coroutine I/O foundation library (Vinnie Falco). https://github.com/cppalliance/capy
5. Corosio - Coroutine networking library (Vinnie Falco). https://github.com/cppalliance/corosio
6. P2300R10 - ''std::execution'' (Michał Dominiak, Lewis Baker, Lee Howes, Kirk Shoop, Michael Garland, Eric Niebler, Bryce Adelstein Lelbach, 2024). https://wg21.link/p2300r10
7. P4058R0 - ''The Case for Coroutines'' (Vinnie Falco, 2026). https://isocpp.org/files/papers/P4058R0.pdf
8. P2430R0 - ''Partial success scenarios with sender/receiver'' (Christopher Kohlhoff, 2021). https://wg21.link/p2430r0
9. P2762R2 - ''Sender/Receiver for Networking'' (Dietmar Kühl, 2023). https://wg21.link/p2762r2
10. P0113R0 - ''Executors and Asynchronous Operations, Revision 2'' (Christopher Kohlhoff, 2015). https://wg21.link/p0113r0
11. P0688R0 - ''A Proposal to Simplify the Unified Executors Design'' (Chris Kohlhoff, Jared Hoberock, Chris Mysen, Gordon Brown, 2017). https://wg21.link/p0688r0
12. P1660R0 - ''A Compromise Executor Design Sketch'' (Jared Hoberock, Michael Garland, et al., 2019). https://wg21.link/p1660r0
13. P1525R0 - ''One-Way execute is a Poor Basis Operation'' (Eric Niebler, 2019). https://wg21.link/p1525r0
14. N3747 - ''A Universal Model for Asynchronous Operations'' (Christopher Kohlhoff, 2013). https://wg21.link/n3747
15. P0285R0 - ''Using customization points to unify executors'' (Christopher Kohlhoff, 2016). https://wg21.link/p0285r0
16. N4370 - ''Networking Library Proposal'' (Christopher Kohlhoff, 2015). https://wg21.link/n4370
17. P0058 - ''An Interface for Abstracting Execution'' (Jared Hoberock, Michael Garland, Olivier Giroux, 2015). https://wg21.link/p0058
18. N4414 - ''Executors and Schedulers'' (Chris Mysen, 2015). https://wg21.link/n4414
19. P1791R0 - ''Evolution of the P0443 Unified Executors Proposal'' (Christopher Kohlhoff, Jamie Allsop, 2019). https://wg21.link/p1791r0
20. P0443R0 - ''A Unified Executors Proposal for C++'' (Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, Carter Edwards, 2016). https://wg21.link/p0443r0
21. N4199 - ''Minutes of Sept. 4-5, 2014 SG1 meeting in Redmond, WA'' (Hans Boehm, 2014). https://wg21.link/n4199

22. P0761R2 - ‘‘Executors Design Document’’ (Jared Hoberock, Michael Garland, Chris Kohlhoff, Chris Mysen, Carter Edwards, Gordon Brown, Michael Wong, 2018). https://wg21.link/p0761r2